2.2 Génération de paysage en 3D

Dans cette section, nous présentons la méthode générale ainsi que les algorithmes utilisés pour la génération de paysages en 3D comprenant une ou plusieurs îles dans **OpenSCAD**. La méthode employée pour générer la matrice de hauteur et construire le paysage diffère légèrement de celle décrite dans l'énoncé du devoir, mais elle produit des résultats similaires tout en intégrant les fonctionnalités de base requises ainsi que certaines fonctionnalités optionnelles pertinentes.

2.2.1 Présentation générale de la méthode :

La génération du paysage en 3D s'effectue en plusieurs étapes successives, voici les grandes lignes de la méthode employée :

- **Initialisation des paramètres** : on définit la taille de la grille, la taille des cubes représentant les hauteurs, ainsi que des paramètres de bruit et de couleur. Un générateur aléatoire est initialisé pour assurer la reproductibilité.
- **Génération du bruit de Perlin** : une carte de hauteur est générée à l'aide du bruit de Perlin, ce qui permet de créer des variations naturelles du relief.
- **Création des îles** : plusieurs centres d'îles sont définis, et un masque circulaire est appliqué pour faire émerger le relief uniquement dans certaines zones du terrain.
- **Lissage du relief** : afin d'obtenir un paysage plus fluide et réaliste, la carte de hauteur est lissée par une méthode itérative.
- **Exportation du modèle** : la carte finale est convertie en primitives OpenSCAD et enregistrée dans un fichier.

2.2.2 Algorithmes :

1. Bruit de Perlin 2D :

Le bruit de Perlin est une fonction continue et pseudo-aléatoire qui permet de générer un champ scalaire aléatoire sur un espace à 2 dimensions, ici noté $f(x, y)$. Il est utilisée pour générer des textures naturelles continues (montagnes, terrain, iles, etc.)

Pour calculer la valeur du bruit de Perlin en un point (x, y), on procède de la facon suivante :

Identification de la grille :

Soit un point $(x, y) \in \mathbb{R}^2$. On commence par localiser la cellule entière de la grille contenant ce point.

On définit :

- les indices entiers de cellule :
$$x_i = \lfloor x \rfloor, \qquad y_i = \lfloor y \rfloor$$
- les positions locales à l'intérieur de la cellule :
$$x_f = x - x_i, \qquad y_f = y - y_i$$

On a donc $x_f, y_f \in [0, 1]$, ce qui représente la position relative du point (x, y) à l'intérieur de la cellule (x_i, y_i) de la grille.

Sélection des gradients aléatoires :

À chaque sommet de la cellule carrée de la grille, on associe un vecteur gradient pseudo-aléatoire, sélectionné selon une fonction de hachage. Les sommets de la cellule sont les suivants :

$$(x_i, y_i), \quad (x_i + 1, y_i), \quad (x_i, y_i + 1), \quad (x_i + 1, y_i + 1)$$

Soit la fonction de permutation pseudo-aléatoire (`get_perm`). La fonction de hachage est définie par :

$$\text{hash}(x, y) = \text{PERM}\left[(\text{PERM}[x \bmod 256] + y) \bmod 256\right]$$

où `PERM` est un tableau de permutation de taille 256 contenant une permutation des entiers de 0 à 255.

Calcul du bruit par produit scalaire :

La fonction `grad(hash, x, y)` utilise la valeur de hachage pour sélectionner l'un des quatre vecteurs directionnels simples :

$$\{(1,1),\ (-1,1),\ (1,-1),\ (-1,-1)\}$$

Pour chaque coin du carré de la grille, on calcule le produit scalaire entre :

- Le vecteur gradient associé au coin,
- Le vecteur reliant ce coin au point considéré (x_f, y_f).

Pour les quatre coins, les produits scalaires sont alors définis comme suit :

$$\begin{aligned}
g_{00} &= \text{grad}(\text{hash}(x_i, y_i),\ \ x_f,\ \ y_f) \\
g_{10} &= \text{grad}(\text{hash}(x_i + 1, y_i),\ \ x_f - 1,\ \ y_f) \\
g_{01} &= \text{grad}(\text{hash}(x_i, y_i + 1),\ \ x_f,\ \ y_f - 1) \\
g_{11} &= \text{grad}(\text{hash}(x_i + 1, y_i + 1),\ \ x_f - 1,\ \ y_f - 1)
\end{aligned}$$

Regardons le schéma pour une cellule particulière :

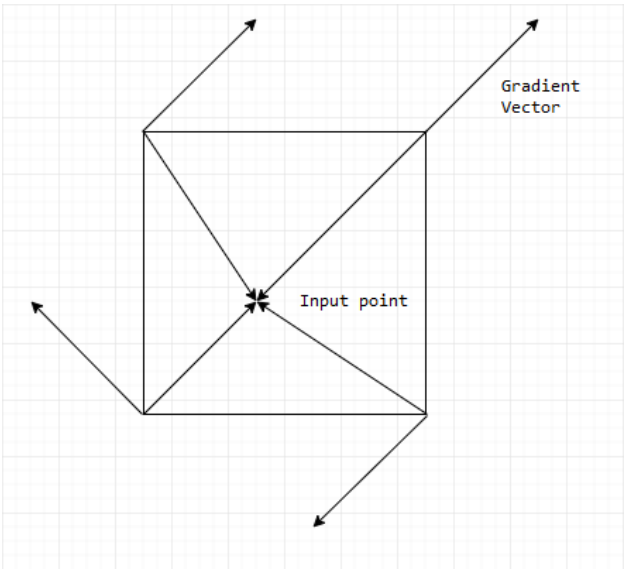

Figure 3: Grille de generation de bruit perlin avec vecteur gradient et vecteur de distance entre le sommet de la cellule et le point entrant. rf : `https://rtouti.github.io/graphics/perlin-noise-algorithm`

Interpolation lissée :

Pour une interpolation douce, on utilise une fonction de lissage `fade`(t) définie comme suit :

$$\text{fade}(t) = 6t^5 - 15t^4 + 10t^3$$

On applique cette interpolation selon x et y,

$$u = \text{fade}(x_f), \quad v = \text{fade}(y_f)$$

où les interpolations linéaires (`lerp`) se définissent par :

$$\text{lerp}(a, b, t) = a + t(b - a)$$

On effectue alors les interpolations selon x pour obtenir :

$$x_1 = \text{lerp}(g_{00}, g_{10}, u), \quad x_2 = \text{lerp}(g_{01}, g_{11}, u)$$

Enfin, l'interpolation finale selon y donne la valeur du bruit :

$$f(x, y) = \text{lerp}(x_1, x_2, v)$$

Bruit fractal multi-octaves :

Pour produire une structure naturelle avec de larges caracteristiques et des details fins, on utilise un bruit fractal multi-octaves. Le bruit fractal est simplement une somme pondérée de bruits de Perlin calculés à différentes fréquences et amplitudes :

$$F(x, y) = \sum_{i=0}^{N-1} \text{Perlin}(x \cdot f_i,\ y \cdot f_i) \cdot a_i$$

Avec :

- N : le nombre d'octaves,
- f_i : la fréquence de l'octave i, typiquement définie par :

$$f_i = f_0 \times (\text{lacunarity})^i$$

- a_i : l'amplitude de l'octave i, typiquement définie par :

$$a_i = a_0 \times (\text{persistence})^i$$

où **persistence** et **lacunarity** sont deux paramètres contrôlant l'aspect fractal :

- **Persistence** : influence l'amplitude des hautes fréquences (les petits détails).
- **Lacunarity** : régule la croissance de la fréquence d'une octave à l'autre.

Ce processus mathématique produit un champ de bruit de Perlin fractal réaliste, continu et régulier. Il est idéal pour la génération de terrains naturels et d'îles procédurales. Cette technique constitue une excellente première étape pour la création de la matrice de hauteur, servant de base à des étapes ultérieures.

Figure 4: Exemple de bruit de Perlin généré sur grille 2D depuis l'étape précédente. rf: Source : `https://rtouti.github.io/graphics/perlin-noise-algorithm`

2. Création des îles :

L'algorithme de création d'îles implémenté permet de générer des îles réalistes à partir d'une graine. L'idée centrale repose sur la modulation de la carte de hauteur initiale en fonction de la distance à un ou plusieurs centres d'île prédéfinis, ce qui permet de sculpter naturellement une ou plusieurs îles dans le champ de bruit.

- La hauteur maximale se situe aux centres.
- La hauteur diminue progressivement vers zéro à mesure qu'on s'éloigne du centre, pour finalement atteindre zéro au rayon maximal.

Formulation mathématique :

Soit une carte de hauteur initiale $H(x, y)$ générée par du bruit fractal, avec des valeurs normalisées typiquement entre -1 et 1. On souhaite définir plusieurs îles, chacune caractérisée par :

- un centre (x_c, y_c),
- un rayon maximal R.

Pour chaque point (x, y) de la carte, on calcule la distance euclidienne au centre de l'île :

$$d = \sqrt{(x - x_c)^2 + (y - y_c)^2}$$

On définit ensuite un **facteur d'île** $f(d)$ qui permet de moduler la hauteur en fonction de la distance au centre. Ce facteur vaut 1 au centre et décroît linéairement jusqu'à 0 au-delà du rayon R :

$$f(d) = \begin{cases} 1 - \dfrac{d}{R} & \text{si } d < R \\ 0 & \text{sinon} \end{cases}$$

Cas de plusieurs centres d'îles :

Lorsque plusieurs centres d'îles sont présents, on combine leurs contributions en prenant le maximum de leurs facteurs respectifs.

Soit N le nombre d'îles, alors pour un point (x, y), le facteur d'île total est défini par :

$$F_{\text{tot}}(x, y) = \max_{1 \leq k \leq N} \left(1 - \frac{d_k}{R_k},\ 0\right)$$

où :

$$d_k = \sqrt{(x - x_{c_k})^2 + (y - y_{c_k})^2}$$

est la distance euclidienne au centre (x_{c_k}, y_{c_k}) de la k-ième île, et R_k est son rayon maximal.

Cette approche garantit que les îles se chevauchent de manière naturelle et continue, en produisant des dégradés de hauteur lisses entre les zones influencées par plusieurs centres.

Normalisation des Hauteur :

On normalise d'abord les valeurs de hauteur de la carte initiale $H(x, y)$, qui sont dans l'intervalle $[-1, 1]$, vers l'intervalle $[0, 1]$ à l'aide de la transformation suivante :

$$H_{\text{norm}}(x, y) = \frac{H(x, y) + 1}{2}$$

On applique ensuite le masque d'île $F_{\text{tot}}(x, y)$ pour obtenir la carte de hauteur finale :

$$H_{\text{final}}(x, y) = H_{\text{norm}}(x, y) \cdot F_{\text{tot}}(x, y)$$

Ce produit permet de conserver la structure du bruit fractal tout en atténuant progressivement les hauteurs vers les bords de l'île pour pouvoir créer des contours plus naturels.

3. Lissage du relief :

Enfin, après avoir généré une carte de hauteur à partir du bruit fractal et avoir appliqué un masque d'île, le relief obtenu peut présenter des transitions abruptes et non réalistes. Le lissage (smoothing) permet de rendre ces transitions plus naturelles, en atténuant les variations brusques des hauteurs entre des points voisins.

L'idée générale de l'algorithme de lissage est assez simple, il repose sur la méthode suivante :

- Remplacer chaque hauteur par la moyenne des hauteurs de ses voisins.
- Répéter ce processus plusieurs fois pour obtenir un lissage progressif.

Formulation mathématique :

Soit $H(x, y)$ la carte de hauteur obtenue après les étapes précédentes, contenant à chaque point (i, j) une hauteur spécifique. La méthode de lissage s'effectue comme suit :

On définit l'ensemble des premiers voisins du point (i, j) par :

$$V(i, j) = \{(x, y) \mid x \in [i-1, i+1],\ y \in [j-1, j+1],\ 0 \leq x < n,\ 0 \leq y < n\}$$

Pour chaque point de la grille, la nouvelle hauteur après une itération est obtenue en prenant la moyenne des hauteurs dans son voisinage $V(i, j)$:

$$H_{\text{lisse}}(i, j) = \frac{1}{|V(i, j)|} \sum_{(x,y) \in V(i,j)} H(x, y)$$

où $|V(i, j)|$ représente le nombre de voisins du point (i, j), qui varie entre 4 et 9 selon sa position dans la grille (coin, bord ou intérieur).

Cette étape est répétée plusieurs fois, selon un nombre d'itérations fixé en fonction du degré de lissage souhaité.

2.2.3 Résultat :

La figure ci-dessous illustre le résultat du générateur de terrain basé sur du bruit de Perlin, combiné avec un masque d'île. Le relief produit est sous forme de cubes unitaires empilés, puis automatiquement exporté au format `OpenSCAD`.

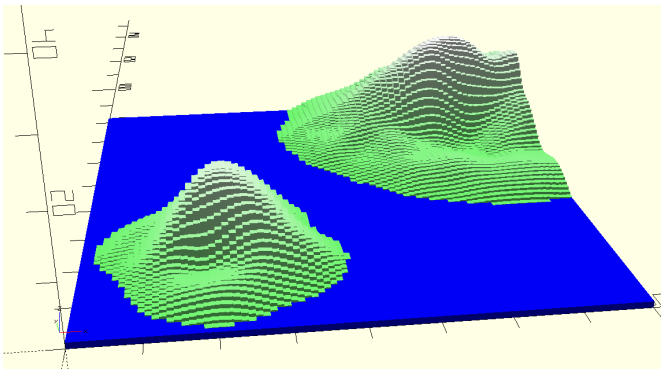

Figure 5: Visualisation 3D du terrain généré par bruit de Perlin et masquage d'îles.

La carte est générée aléatoirement à partir de deux centres d'îles placés de facon arbitraire. Les rayons maximaux associés à ces centres sont fixés.

```
# Defini le centre et le rayon des îles
island_centers = [(N / 4, N / 4), (7 * N / 8, 7 * N / 8)]
island_radii = [N / 4, N / 2]
```

La carte est générée de façon procédurale et aléatoire, mais une fois qu'une configuration satisfaisante est obtenue, la graine (*seed*) du générateur aléatoire peut être fixée afin d'assurer la reproductibilité du résultat.

2.2.4 Fonctionnalités supplémentaires :

1. Maillage hexagonal :

On introduit ici notre premiere fonctionnalité supplémentaire, le remplacement des cubes unitaires par des prismes hexagonaux. Un hexagone régulier est défini par 6 sommets également espacés. Les sommets sont calculés une seule fois pour tous les prismes et sont ensuite convertie pour la fonction `polygon()` de OpenSCAD.

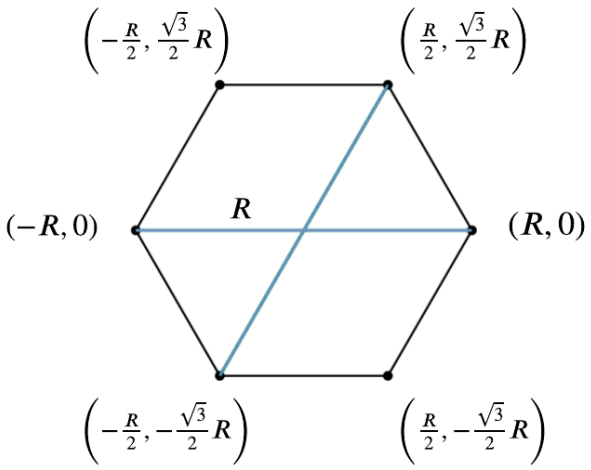

Figure 6: Géométrie et sommets de l'hexagone (flat-topped hexagon). rf : red-blobgames.com/grids/hexagons

Soit $R =$ `hex_size` le rayon de l'hexagone (distance entre le centre et un sommet). Les six sommets relatifs au centre sont donnés par :

$$P_k = (R\cos\theta_k,\ R\sin\theta_k)\,, \quad \theta_k = \frac{k\pi}{3}, \quad k = 0, 1, \ldots, 5.$$

Ou de facon explicite,

$$\{P_k\} = \left\{(R, 0),\ \left(\frac{R}{2}, \frac{\sqrt{3}}{2}R\right),\ \left(-\frac{R}{2}, \frac{\sqrt{3}}{2}R\right),\ (-R, 0),\ \left(-\frac{R}{2}, -\frac{\sqrt{3}}{2}R\right),\ \left(\frac{R}{2}, -\frac{\sqrt{3}}{2}R\right)\right\}$$

Pour disposer les hexagones en grille (flat-topped), on définit les centres :

$$x_{i,j} = \frac{3R}{2}\,i, \quad y_{i,j} = \sqrt{3}\,R\,j + (i \bmod 2)\cdot\frac{\sqrt{3}}{2}\,R.$$

- L'espacement horizontal entre centres vaut $\frac{3R}{2}$.
- L'espacement vertical de base vaut $\sqrt{3}R$.
- On ajoute un décalage de $\frac{\sqrt{3}}{2}R$ pour chaque colonne i impaire.

2. Gradient de couleur :

La seconde fonctionnalité supplémentaire est l'implémentation d'un gradient de couleur tres similaire a celui presenté dans l'exemple de l'enoncé. Cette fonctionnalité associe une couleur a chaque hauteur selon un dégradé progressif.

Pour une valeur de hauteur $h \in [h_{min}, h_{max}]$, on calcule d'abord un rapport normalisé :

$$r = \frac{h - h_{min}}{h_{min} - h_{max}} \in [0, 1]$$

Cette normalisation permet de transformer les hauteurs arbitraires en un rapport utilisable pour l'interpolation.

Avant d'interpoler les couleurs, on modifie ce rapport par un facteur de dégradé $\gamma > 0$ qui permet de controler la rapidité du dégradé de couleur :

$$r_{mod} = r^{\gamma}$$

- Si $\gamma = 1$: dégradé linéaire
- Si $\gamma < 1$: les couleurs s'éclaircissent plus vite (exagère les niveaux bas)
- Si $\gamma > 1$: les couleurs changent plus lentement (met l'accent sur les hauts reliefs)

On peut alors effectuer l'interpolation, on définit deux couleurs de base :

- Couleur minimale : $\vec{C}_{\text{bas}} = (r_1, g_1, b_1)$
- Couleur maximale : $\vec{C}_{\text{haut}} = (r_2, g_2, b_2)$

Et on interpole linéairement chaque composante :

$$\vec{C}(h) = (1 - r_{\text{mod}})\,\vec{C}_{\text{bas}} + r_{\text{mod}}\,\vec{C}_{\text{haut}}$$

Ou coordonnée par coordonnée :

$$\begin{aligned} R(h) &= r_1 + (r_2 - r_1) \cdot r_{\text{mod}} \\ G(h) &= g_1 + (g_2 - g_1) \cdot r_{\text{mod}} \\ B(h) &= b_1 + (b_2 - b_1) \cdot r_{\text{mod}} \end{aligned}$$

3. Generation d'arbres probabilistes :

Paramètres du Code

- `height_map` : Matrice $n \times n$ des altitudes
- `tree_density` ($\rho = 0.1$) : Probabilité de générer des arbres sur une cellule côtière
- `coastal_range` ($r = 5$) : Rayon maximal de placement autour des côtes
- `min_height` ($h_{\min} = 0.1$) : Seuil eau/terre

Processus Aléatoire

Étape 1 : Détection Côtière Pour chaque cellule (i, j) :

$$\text{Si } \texttt{height_map}[i][j] > h_{\min} \text{ et qu'au moins une des 8 cases autour est de l'eau}$$

Plus précisément :

$$\exists (di, dj) \in \{-1, 0, 1\}^2 \text{ avec } (di, dj) \neq (0, 0) \text{ tel que } \texttt{height_map}[i + di][j + dj] \leq h_{\min}$$

Étape 2 : Génération d'Arbres Pour chaque cellule côtière (i, j) :

1. Tirage de génération :

$$G \sim \text{Bernoulli}(\rho) \quad \text{(1 si arbres générés, 0 sinon)}$$

2. Si $G = 1$:
   - Nombre d'arbres $k \sim \mathcal{U}\{1, \ldots, 5\}$
   - Pour chaque arbre :

$$\begin{aligned} di &\sim \mathcal{U}\{-r, \ldots, r\} \\ dj &\sim \mathcal{U}\{-r, \ldots, r\} \\ (neighbor_i, neighbor_j) &= (i + di, j + dj) \end{aligned}$$

   - Validation :

     $$\text{Si } 0 \leq neighbors_i, neighbors_j < n \text{ et } \texttt{height_map}[neighbor_i][neighbor_j] > h_{\min} \Rightarrow \text{Ajout à } \texttt{trees}$$

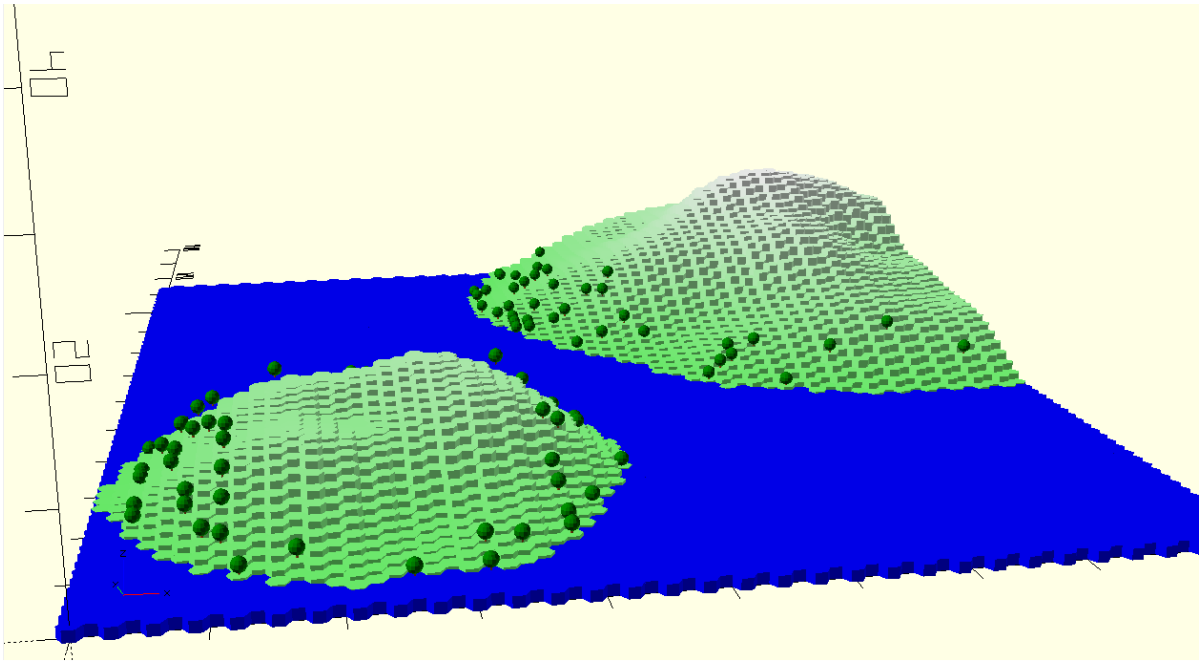

Figure 7: Rendu du paysage final

Les quatre fonctionnalités supplémentaires ajoutées sont :

1. Ajout de bruit aléatoire dans l'élévation.
2. Utilisation des couleurs dans l'export OpenSCAD.
3. Génération d'îles dont la forme n'est pas un cercle parfait ni une ellipse simple.
4. Utilisation de tuiles hexagonales à la place de carrés.
5. Génération d'arbres probabilistes en région côtière.